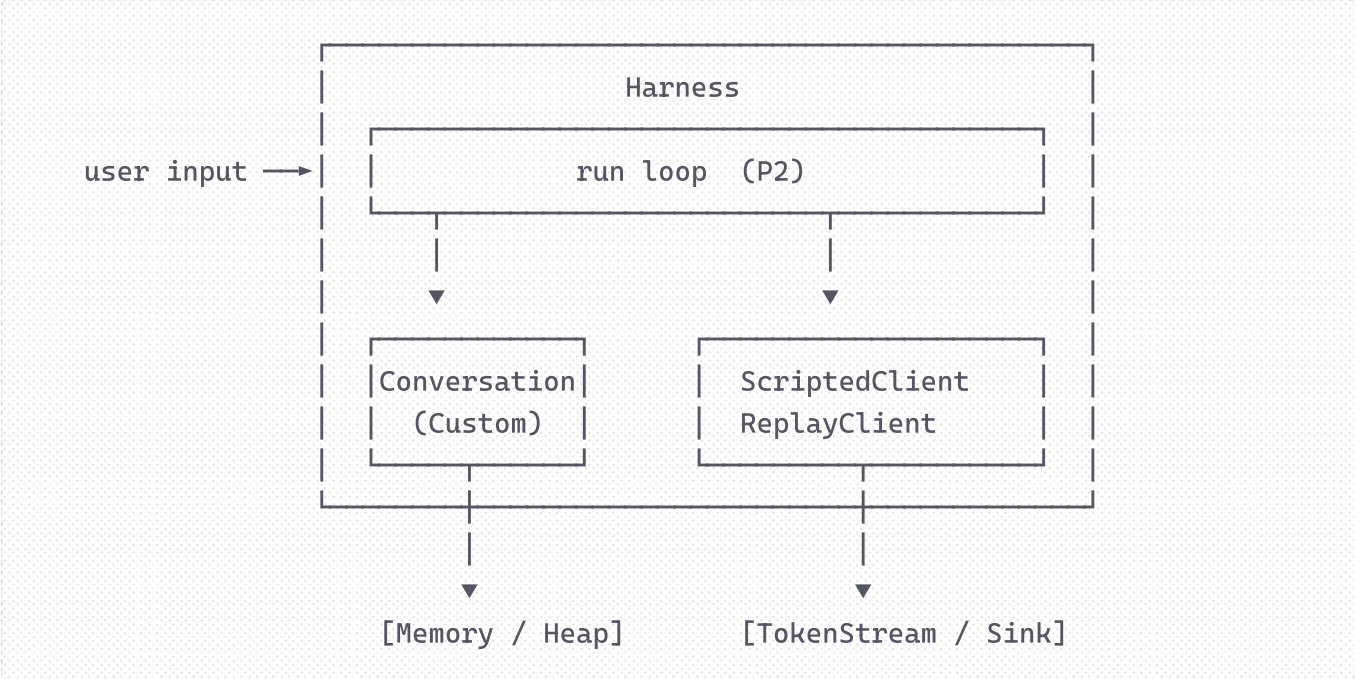
ECE 309: Project 2 — The Conversation Loop

Weight: 15% of project grade | **Duration:** Weeks 2–4

1. Project Overview & Learning Goals

An LLM harness is the program that sits between a language model and the outside world. A raw language model can only produce text. The harness is what turns that text into an agent: it maintains conversation history, decides when a session is over, handles token streaming, and manages user input loops.

In Project 2, you will build the core memory and streaming components for the execution loop (`miniharness`) in C++.



Learning Goals:

- Implement a sequence container (growable array) and reason rigorously about its amortized cost — this is the central exercise of the project.
- Manage object lifetime with the Rule of Five: deep-copy semantics, pointer-stealing move semantics, and exception-safe destruction.
- Solve a real streaming problem: detect a multi-character sentinel in a character stream that arrives in arbitrary chunks, using bounded memory.
- Read and correctly use a polymorphic interface (`ModelClient`) and a provided execution loop (`Harness`) without modifying either.

What's provided vs. what's yours. Both `ModelClient` implementations (`ScriptedModelClient`, `ReplayModelClient`) and the full `Harness::run()` loop — including `main.cpp` and the terminal I/O — are given to you as working, compiled starter code. In a real harness, `ModelClient` would wrap an HTTP call to a model API that hands you characters back; here it's a `.script` file that does the same job, so your tests stay deterministic. Your job is `Conversation` and `SentinelScanner`. Read §3.3 and §3.5 to understand what the provided code expects from you — not to reimplement it.

---

2. Program Behaviors & Requirements

Your executable, `miniharness`, starts a conversation, alternates turns between a user and a model, and stops when the model emits the end-of-conversation sentinel `<|end_conversation|>` or when a turn limit is reached.

Example CLI Interaction:

```
$ ./miniharness --script scripts/greeting.script
you> hello
assistant> Hi! What can I do for you today?
you> nothing, bye
assistant> Goodbye.
[conversation ended: stop sentinel after 2 turns]
```

Required Behaviors:

1. **Roles:** Every message carries a role: `System`, `User`, or `Assistant`. A system message, if present, is always first and is never evicted.
2. **Two Model Clients:**
   - `ScriptedModelClient` reads a `.script` file and returns the next scripted reply sequentially.
   - `ReplayModelClient` reads a previously recorded transcript and replays assistant turns verbatim. This is used by the test harness.
3. **Streaming & Sentinels:** The model interface must support delivering a reply in chunks. Chunk boundaries are arbitrary and *may split the sentinel* (e.g., `"Goodbye.<|end_"` followed by `"conversation|>"`). Your loop must terminate correctly and must not print the sentinel to the user.
4. **Turn Limit:** Passing `--max-turns N` (default 20) stops the loop with a distinct exit reason.
5. **Transcript Output:** Passing `--save transcript.txt` writes the full conversation to disk, such that feeding it back through `ReplayModelClient` reproduces the session exactly.

6. **Clean Shutdown:** Pressing Ctrl-D (EOF) on standard input ends the conversation gracefully and still writes the transcript.

Note on command-line arguments. The provided `main.cpp` fully handles parsing all command-line flags (`--max-turns` , `--script` , `--save`). You do not need to write the CLI parsing logic — your focus is strictly on making sure `Conversation` and `SentinelScanner` behave correctly when the provided loop uses them.

---

3. Architecture & Class Design

The table below says who writes what. For classes marked **provided**, the header *and* implementation are given in the starter repo — read them, don't rewrite them. For classes marked **you implement**, only the public interface is fixed; you choose the private representation.

Class	Status
`Message`	You implement (trivial)
`Conversation`	**You implement — the core of P2**
`SentinelScanner`	You implement
`ModelClient` , `TokenSink`	Provided
`ScriptedModelClient` , `ReplayModelClient`	Provided
`Harness` , `InputSource` , `OutputSink` , `main.cpp`	Provided

3.1 The `Message` Class (`core/message.h`)

Messages must carry a role and string content. Because your conversation history will allocate an array of these, you must provide a default constructor.

```
enum class Role { System, User, Assistant };

class Message {
public:
    // Default-constructs an empty System message with empty content.
    // Needed so Conversation can allocate raw array slots before
    // append() fills them in.
    Message();

    Message(Role role, std::string content);
```

```
    Role               role()    const noexcept;  // Who sent this message.
    const std::string& content() const noexcept;  // The message text.

private:
    Role        role_;
    std::string content_;
};
```

3.2 The `Conversation` Container (`core/conversation.h`)

You must implement your own growable array. **The use of `std::vector` is strictly forbidden.**

This class is the *only* place in your entire codebase where raw `new` and `delete` are permitted.

```
class Conversation {
public:
    // Empty conversation: size() == 0, no allocation yet.
    Conversation();

    // Releases all owned Message storage. No effect if already empty
    // (e.g. moved-from).
    ~Conversation();

    // Deep copy: allocates its own buffer and copies every Message.
    // this->begin() must differ from other.begin() afterward.
    Conversation(const Conversation& other);
    Conversation& operator=(const Conversation& other);

    // Steals other's buffer - no per-element copying. Afterward, other
    // must be left valid and empty (safe to destroy or reassign).
    Conversation(Conversation&& other) noexcept;
    Conversation& operator=(Conversation&& other) noexcept;

    // Appends m, growing the backing array if needed. Amortized O(1) -
    // document and justify your growth strategy in the design log
    // (see Appendix C if you want a refresher first).
    void append(Message m);

    // Number of messages currently stored.
    std::size_t size() const noexcept;

    // Bounds-checked access. Decide what happens on i >= size() (throw,
    // assert, whatever you pick) and test that behavior explicitly.
    const Message& at(std::size_t i) const;
```

```
    // Range-for iteration, oldest message first. begin() == end() when
    // size() == 0.
    const Message* begin() const noexcept;
    const Message* end()   const noexcept;

private:
    Message*    data_ = nullptr;
    std::size_t size_ = 0;
    std::size_t capacity_ = 0;
};
```

- **Memory Management:** You must obey the Rule of Five. Your copy semantics must perform deep copies. Your move semantics must steal the pointer and zero out the source object.
- **Amortized Cost:** You must document your chosen growth factor and prove the amortized $O(1)$ cost of the `append` operation in your Design Log.

3.3 The Model Interface (`model/model_client.h`) — Provided

`ModelClient` and its two implementations are given, fully working, in the starter repo. In a real harness this wraps an HTTP call to a model API that streams characters back; here `ScriptedModelClient` reads a `.script` file and `ReplayModelClient` replays a transcript, so your tests stay deterministic. You call these classes; you do not modify or subclass them.

```
struct StopReason {
    enum class Kind { Sentinel, TurnLimit, UserExit, ClientError } kind;
    std::string detail;
};

class TokenSink {
public:
    virtual ~TokenSink() = default;
    virtual void on_chunk(std::string_view chunk) = 0;
    virtual void on_complete() = 0;
};

class ModelClient {
public:
    virtual ~ModelClient() = default;
    virtual void generate(const Conversation& conv, TokenSink& sink) = 0;
    Message generate(const Conversation& conv);
};
```

Sample usage — roughly what the provided `Harness::run()` does internally:

```
class PrintingSink : public TokenSink {
public:
    void on_chunk(std::string_view chunk) override { std::cout << chunk; }
    void on_complete() override { std::cout << "\n"; }
};

ScriptedModelClient model("scripts/greeting.script");
Conversation conv;
conv.append(Message(Role::User, "hello"));

PrintingSink sink;
model.generate(conv, sink);   // streams the reply through
on_chunk/on_complete
```

(Optional background: if you're curious *why* `ModelClient` is shaped this way, see Appendix B. Not needed to complete P2.)

3.4 The `SentinelScanner` (`core/sentinel_scanner.h`)

The model streams its reply in arbitrary-sized pieces, and the stop sentinel `<|end_conversation|>` can land anywhere relative to those piece boundaries. `SentinelScanner` incrementally tracks "is any of what I've seen so far the start of the sentinel?" without holding the whole reply in memory.

Your scanner needs to handle three shapes of input correctly:

1. **Whole sentinel in one chunk** — `feed("Goodbye.<|end_conversation|>")` emits `"Goodbye."` as safe text and reports `sentinel_found = true`.
2. **Sentinel split at one arbitrary point across two chunks** — e.g. `feed("Goodbye.<|end_")` then `feed("conversation|>")`. The first call must not emit the partial sentinel as safe, and must not falsely report a match yet.
3. **Sentinel arriving one character at a time** — same as (2), split at every possible boundary. This is what the autograder stress-tests.

```
class SentinelScanner {
public:
    explicit SentinelScanner(std::string sentinel);

    struct Out { std::string safe_text; bool sentinel_found; };

    // Feed the next chunk. Returns text guaranteed NOT to be part of
    // the sentinel (safe to print immediately) and whether the
    // sentinel has now been fully seen.
```

```
    Out feed(std::string_view chunk);

    // Call once, after the stream ends, to release any text still
    // being held back.
    Out flush();

private:
    std::string sentinel_;
    std::string pending_;   // holds back at most sentinel_.size() - 1
                            // trailing characters that could still
                            // become the start of the sentinel
};
```

The approach that satisfies all three scenarios: keep at most `sentinel_.size() - 1` trailing characters in `pending_` at all times. On each `feed`, treat `pending_ + chunk` as the text to search; if the sentinel isn't found, everything except the last `sentinel_.size() - 1` characters is safe to emit, and the remainder becomes the new `pending_`. Prove in your design log that `pending_` never grows past that bound — that bound is what makes this O(1) space per chunk instead of the O(N²) blowup from concatenating everything and re-searching from scratch.

Stretch goal, not required: replace this with Knuth–Morris–Pratt and measure the difference on adversarial input like `<|end_<|end_<|end_...`.

3.5 The `Harness` (`harness/harness.h`) — Provided

The full turn loop — reading user input, calling the model, running the reply through `SentinelScanner`, appending both turns to the `Conversation`, checking the turn limit, handling EOF — is given as working code, along with `main.cpp` and terminal `InputSource` / `OutputSink` implementations. Read it: it's the reference for how `Conversation` and `SentinelScanner` are actually used together.

```
struct HarnessConfig {
    int max_turns = 20;
    std::string system_message;

};

class InputSource {
public:
    virtual ~InputSource() = default;
    virtual std::string read_line() = 0;
    virtual bool is_eof() const = 0;
};
```

```
class OutputSink {
public:
    virtual ~OutputSink() = default;
    virtual void write(std::string_view text) = 0;
};

class Harness {
public:
    Harness(std::unique_ptr<ModelClient> model, HarnessConfig cfg);
    StopReason run(InputSource& in, OutputSink& out);

private:
    std::unique_ptr<ModelClient> model_;
    Conversation                 conv_;
    HarnessConfig                cfg_;
};
```

---

4. Common Pitfalls

Review these common mistakes before submitting, as they are the most frequent causes of failed test cases:

1. **Ownership Issues:** Storing `Message*` in the conversation and letting the caller keep ownership. Ownership must be unambiguous.
2. **Shallow Copies:** A `Conversation` copy constructor that copies the pointer instead of the buffer. This causes a double-free at scope exit which AddressSanitizer will immediately flag.

---

5. Deliverables & Testing

Your repository must be built with CMake and compile cleanly under AddressSanitizer (`-fsanitize=address`).

Folder Structure:

- `src/` (Source files)
- `include/` (Headers)
- `tests/` (Test suite)
- `docs/` (Design log)

Repository Structure

```
├── CMakeLists.txt
├── include/
│   ├── core/
│   │   ├── conversation.h
│   │   ├── message.h
│   │   └── sentinel_scanner.h
│   ├── harness/
│   │   └── harness.h
│   └── model/
│       ├── model_client.h
│       ├── scripted_client.h
│       └── replay_client.h
├── src/
│   ├── conversation.cpp
│   ├── sentinel_scanner.cpp
│   ├── model_client.cpp
│   ├── scripted_client.cpp
│   ├── replay_client.cpp
│   ├── harness.cpp
│   └── main.cpp
├── tests/
│   └── p2/
│       └── test_p2.cpp
└── docs/
    └── design-log-p2.md
```

Test Suite Requirements (`tests/p2/test_p2.cpp`):
You must write at least 12 assert-based test cases. Weight your effort toward `Conversation` and `SentinelScanner` — that's your code. The `Harness`-related items confirm you've wired the provided pieces correctly, not logic you wrote yourself.

1. **Empty Conversation Bounds:** Handle empty conversations without out-of-bounds access.
2. **System Message Ordering:** Ensure system messages remain pinned at the front.
3. **Rule of Five (Copy):** Assert copy constructors allocate entirely different pointer addresses.
4. **Rule of Five (Move):** Assert move constructors steal the data pointer and zero the source.
5. **Growth behavior:** Assert capacity grows per your documented growth factor and `size()` / `at()` stay correct across reallocation.
6. **Scanner (Clean Text):** Verify the scanner processes strings with no sentinel correctly.
7. **Scanner (Split Sentinel):** Prove the scanner catches the sentinel split across *every possible boundary* (loop over all split points programmatically).
8. **Scanner (False Alarms):** Ensure the scanner doesn't trigger on partial matches (e.g., `<|end_world|>`).

9. **Scanner (Bounded Memory):** Assert `pending_` never exceeds `sentinel.size() - 1` while feeding a large adversarial stream.
10. **Harness (Turn Limit):** Confirm the provided loop stops with `TurnLimit` when your `Conversation` is used underneath it.
11. **Harness (Sentinel Halt):** Confirm the provided loop halts exactly when your `SentinelScanner` reports the sentinel found.
12. **Transcript Round-Trip:** Save a mock conversation, load it via the provided `ReplayModelClient`, assert identical playback.

Sample test, showing the expected rigor for item 7:

```
TEST(ScannerCatchesSentinelAtEveryBoundary) {
    const std::string sentinel = "<|end_conversation|>";
    const std::string text = "Goodbye." + sentinel;
    for (std::size_t split = 0; split <= text.size(); ++split) {
        SentinelScanner scanner(sentinel);
        auto out1 = scanner.feed(text.substr(0, split));
        auto out2 = scanner.feed(text.substr(split));
        assert((out1.sentinel_found || out2.sentinel_found) &&
               "sentinel must be caught regardless of split point");
        assert(out1.safe_text + out2.safe_text == "Goodbye.");
    }
}
```

Design Log (`docs/design-log-p2.md`):
A 500–800 word Markdown document defending your design. It must cover:

- Your growth factor choice and the proof of amortized $O(1)$ insertions.
- Evidence of how your code handles the Rule of Five safely.
- The mathematical argument proving your pending-buffer never exceeds the sentinel length.
- One thing you would design differently in hindsight.

Prepare Final Submission Files: Create the following two files to submit for grading:

- **github.txt:** A simple text file containing the direct URL to your GitHub repository.
- **github.zip:** A compressed ZIP file containing your entire repository as a backup.

---

6. Grading Rubric

Criterion	Points	Evaluation
Rule of Five & Memory Safety	35	Automated tests under AddressSanitizer (0 leaks); deep-copy and steal-on-move verified.
Amortized Growth	10	Design log: growth factor documented and amortized O(1) proven.
Sentinel Bounded Memory	25	Stress test feeding a 4MB stream one byte at a time; `pending_` bound verified.
Correct Use of Provided Harness/ModelClient	10	Tests confirm turn limit, EOF, and sentinel-halt behavior when your classes run under the provided loop.
Test Suite Quality	10	Manual review of your test cases.
Design Log	10	Evaluation of your amortized math and buffer proofs.

7. Appendix A — Transcript and Script Format

(Reference only — this format is produced/consumed by the provided `ScriptedModelClient` / `ReplayModelClient` / `Harness`. You don't need to write a parser for it.)

The `.script` files (input) and `transcript.txt` files (output) are line-oriented, with one message per block. Blocks are separated by a line containing exactly `---`.

- **Content Escaping:** A line containing exactly `---` is strictly reserved for message boundaries. A message's text content cannot contain a bare `---` line.
- **System Messages:** If a script or transcript begins with a block tagged `role: system`, that becomes the initial system message.

Standard Transcript Format

Used for `--save` outputs and `ReplayModelClient` inputs.

```
role: system
Be concise.
---
role: user
hello
---
role: assistant
Hi! What can I do for you today?
---
role: user
```

```
goodbye
---
role: assistant
Goodbye.<|end_conversation|>
```

Script Format (`ScriptedModelClient`)

A `.script` file is similar to a transcript but can include directives to test your harness's streaming logic. **Directives must appear *before* the `role:` declaration in any block, in any order.**

```
match: /file|todo/i
chunk: 5
role: assistant
This will stream 5 characters at a time.<|end_conversation|>
```

- `chunk: N` — Forces the `ScriptedModelClient` to emit the reply in chunks of `N` characters. This is how you test that your `SentinelScanner` correctly handles sentinels split across chunks. *If omitted, the default behavior is to emit the entire message as a single chunk.*
- **Block Exhaustion:** If the `ScriptedModelClient` is asked to generate a reply but has exhausted all of its available scripted blocks, it must trigger a loop termination and yield a `ClientError` StopReason.
- `match: /pattern/i` — *(Optional Stretch Goal)* Allows the scripted client to select a reply by matching a regex pattern against the most recent user message. Assume the default `std::regex` flavor (ECMAScript) is used. **Semantics:** Search proceeds top-to-bottom through the unconsumed blocks. The first block whose pattern matches wins. A block without a `match` directive acts as an unconditional catch-all default. If you do not implement branching, your client can simply ignore `match` directives and read the blocks sequentially.

8. Appendix B — Optional Background: Abstract Base Classes and Virtual Functions

You don't need this to complete P2 — `ModelClient` and its subclasses are provided. Read it if you want to understand *why* the provided code is shaped the way it is.

A pure virtual function (`virtual void generate(...) = 0;`) has no body in the class that declares it — it just states "every subclass must implement this." A class with even one pure virtual function is *abstract*: you cannot write `ModelClient m;` , only `ScriptedModelClient s;` or `ReplayModelClient r;` , each providing its own body for `generate` .

`override` on a subclass's method is a compiler check, not a requirement of the language — it just confirms you're actually replacing a virtual function with a matching signature, rather than accidentally declaring a new, unrelated one.

`ModelClient` also has a second, non-virtual `generate(const Conversation&)` that calls the virtual one internally. Because C++ hides *all* overloads of a name once you override any of them, a subclass that overrides the virtual `generate` makes the non-virtual one invisible from outside — unless you write `using ModelClient::generate;` in that subclass to bring it back into scope. This pattern (virtual low-level function, non-virtual convenience wrapper in the base) is called the Non-Virtual Interface idiom, and it's why you'll see that `using` line in the spec.

9. Appendix C — Primer: Growable Arrays and Growth Factor

A growable array like `Conversation` starts with some `capacity_` and, when `append` is called with `size_ == capacity_`, must allocate a bigger buffer, move the existing elements over, and free the old one. The *growth factor* is how much bigger the new buffer is — double it (`new_capacity = capacity_ == 0 ? 1 : capacity_ * 2`), grow by 1.5x, or something else.

Why not grow by a fixed amount (e.g., +1 each time)? Every `append` would then trigger a reallocation, and each reallocation copies all existing elements — for `n` appends that's `1 + 2 + ... + n = O(n^2)` total copying.

Doubling avoids this: the reallocation at size `k` copies `k` elements, but is followed by roughly `k` more appends before the next reallocation (since capacity just doubled). The copying cost of each reallocation is "paid for" by the appends that follow it, so the total copying across `n` appends is `O(n)`, not `O(n^2)` — `O(1)` amortized per `append`. You'll need to write this argument out precisely for your design log, and you're free to pick a factor other than exactly 2 — just justify it and prove the bound holds for whatever you choose.